

Project overview

This app is a Community Safety & Incident Tracker built with:

- Backend: `Node.js`, `Express`, `better-sqlite3`, `jsonwebtoken`, `bcryptjs`, `multer`
- Frontend: `plain HTML`, `CSS`, `JavaScript`
- Data: SQLite DB in `app.db`
- Live alerts: Server-Sent Events (SSE)

It supports:

- login by role (`admin`, `analyst`, `viewer`)
- incident creation, editing, deletion
- attachments upload
- live alert ticker
- internal messaging
- export/import incident data
- audit logging

Complete explanation of the code

1. `server.js`

This is the main app entry point.

What it does:

- loads env vars with `dotenv`
- creates `data` and `uploads`
- serves `public`
- mounts API routes:
 - `/api/auth`
 - `/api/incidents`
 - `/api/messages`
 - `/api/alerts`
 - `/api/audit`
 - `/api/export`
 - `/api/import`
- sets up SSE at `/api/alerts/stream`
- periodically generates automated alerts every 2 minutes
- serves `index.html` for all other requests

Key behavior:

- SSE clients are stored in `sseClients`
- `app.on('new_alert', ...)` broadcasts alerts to all connected browser clients

PROJECT TASK DISTRIBUTION

2. `# database.js`

Initializes the SQLite database and seeds default users.

Tables created:

- `users`
- `incidents`
- `alerts`
- `messages`
- `audit_logs`

Default account seeds:

- `admin / admin123`
- `analyst / analyst123`
- `viewer / viewer123`

Example:

```
const defaultUsers = [
  { username: 'admin', password: 'admin123', role: 'admin' },
  { username: 'analyst', password: 'analyst123', role: 'analyst' },
  { username: 'viewer', password: 'viewer123', role: 'viewer' }
];
```

3. `# auth.js`

Handles authentication and role authorization.

Routes use these guards:

- `requireAuth` -> checks JWT Bearer token
- `requireAdmin` -> only admin allowed
- `requireAnalystOrAdmin` -> analyst or admin allowed

This is the backbone of security for protected endpoints.

4. `# auth.js`

Manages login and current user info.

- `POST /api/auth/login`
 - checks username/password
 - returns JWT token
 - logs login to `audit_logs`
- `GET /api/auth/me`
 - returns current user data
- `POST /api/auth/logout`
 - logs logout event

The route uses:

```
const token = jwt.sign({ id: user.id, username: user.username, role: user.role }, JWT_SECRET, { expiresIn: '8h' });
```

5. incidents.js

Handles incident management and attachments.

Routes:

- GET /api/incidents
 - returns incidents
 - filters by severity/status/search
 - limits viewer to visibility = public
- POST /api/incidents
 - create incident
 - requires analyst or admin
 - supports file upload via multer
- PUT /api/incidents/:id
 - update incident
 - requires analyst or admin
- DELETE /api/incidents/:id
 - admin only
- DELETE /api/incidents/batch
 - admin only batch delete

Attachments are stored in uploads and metadata is saved as JSON in the attachments field.

6. alerts.js

Alert history and manual alert simulation.

- GET /api/alerts
 - fetches recent alerts
- POST /api/alerts/simulate
 - admin-only manual alert creation
 - emits new_alert to SSE

This is how the live alert ticker works.

7. messages.js

Internal messaging system.

- GET /api/messages/inbox
 - returns messages for current user
 - includes broadcast messages (recipient_id IS NULL)
- GET /api/messages/users
 - returns user list for recipient selection
- POST /api/messages
 - send message
 - recipient_id === 'broadcast' becomes null for broadcast

8. `export.js`

Export and import functionality for incidents.

- `GET /api/export/json`
 - downloads incidents as JSON
- `GET /api/export/csv`
 - downloads incidents as CSV
- `POST /api/export/json`
 - imports JSON incidents and replaces existing records

Admin-only for export/import.

9. `script.js`

Frontend logic for login, dashboard, alerts, incidents, messages, export, and theme.

Highlights:

- stores JWT token in `sessionStorage`
- calls `/api/auth/me` on load
- toggles UI elements by role:
 - `.admin-only`
 - `.analyst-admin-only`
- opens modals for new incident, edit incident, compose message, simulate alert
- uses `EventSource('/api/alerts/stream')` for live alerts
- renders incident rows and buttons based on user role
- sends requests with `Authorization: Bearer <token>`

Example:

```
function getAuthHeaders(isFileUpload = false) {
  const token = sessionStorage.getItem('token');
  const headers = {};
  if (token) headers['Authorization'] = `Bearer ${token}`;
  if (!isFileUpload) headers['Content-Type'] = 'application/json';
  return headers;
}
```

Recommended team task distribution

Member 1 — Backend & Authentication

Files:

- `server.js`
- `auth.js`
- `auth.js`
- `incidents.js`
- `database.js`

Responsibilities:

- implement login + JWT auth
 - implement incident CRUD and role restrictions
 - ensure uploads work
 - verify route behavior with `admin`, `analyst`, `viewer`
-

Member 2 — Frontend UI / Dashboard

Files:

- `index.html`
- `style.css`
- `script.js`

Responsibilities:

- build/clean up dashboard layout
 - implement theme toggle, navigation, modals
 - render incidents/messages/alerts
 - show/hide controls by role
 - make UI responsive and easy to demo
-

Member 3 — Alerts, Messaging, Export/Import

Files:

- `alerts.js`
- `messages.js`
- `export.js`
- supporting frontend sections in `script.js`

Responsibilities:

- implement alerts stream + alert simulation
- implement message inbox & sending
- implement export CSV/JSON and import JSON
- demonstrate live alert ticker and broadcast messaging

Member 4 — QA, Presentation, Documentation

Files:

- `📄 README.md`
- demo flow and test cases
- final presentation preparation

Responsibilities:

- prepare demo script
- test all roles and major workflows
- validate data export/import and SSE alerts
- document default credentials, setup steps, and features
- answer professor questions about architecture and team roles

Demo script to show the professor

1. Start the app:

- `npm install`
- `npm start`
- open `http://localhost:3000`

2. Login as `admin / admin123`

- show admin-only controls
- create a new incident
- simulate an alert
- export incidents to CSV / JSON

3. Login as `analyst`

- show incident editing
- show that analyst can create and update but not delete
- send a message to another user

4. Login as `viewer`

- show limited view
- show public incident visibility only
- confirm no edit/delete buttons

5. Show live alert ticker

- trigger an alert
- show it appears instantly across clients if available

6. Mention backend flow:

- auth with JWT
- SQLite database schema
- role-based route guards in `📄 auth.js`

Difference Between CSV and JSON

Feature	CSV	JSON
Full Form	Comma-Separated Values	JavaScript Object Notation
Data Format	Rows and columns	Key-value pairs
Complexity	Simple	More structured
Supports Nested Data	No	Yes
Readability	Easy for tables	Easy for structured data
File Size	Smaller	Larger
Mostly Used In	Spreadsheets, databases	APIs, web apps

Prompt for Antigravity

Build a complete full-stack web application called **Community Safety & Incident Tracker**.

This is a simple and practical neighborhood safety reporting system for residents, volunteers, and local admins to report, monitor, and manage local incidents.

The project must be **simple, stable, beginner-friendly, and easy to run locally**.

All persistent data must be stored on the **Node.js backend** using **SQLite (better-sqlite3)**.

Do **not** store application data in browser localStorage (except JWT token and theme preference).

The application must be fully functional, cleanly structured, and ready to run after installation.

Main Goal

Build a simple but effective community safety reporting system where users can:

- log in securely
- create and manage incidents
- upload evidence files
- receive live safety alerts
- send secure internal messages
- view audit logs
- export/import incident data
- use role-based access control

The project must prioritize:

- simplicity
- stability
- clean backend
- minimal dependencies
- easy setup
- smooth local execution

Avoid overengineering.

Do not use unnecessary libraries or advanced features that may cause setup or runtime issues.

Tech Stack (Use Exactly This)

Backend

- Node.js
- Express.js
- better-sqlite3
- bcrypt
- jsonwebtoken
- multer
- dotenv

Frontend

- HTML
- CSS
- Vanilla JavaScript

Database

- SQLite using **better-sqlite3**

Real-Time

- Server-Sent Events (SSE) only

Important Simplifications (Must Follow)

To keep the project stable and easy to run:

1. **Do not use 2FA**
2. **Do not use Nodemailer**
3. **Do not use WebSockets**
4. **Do not use Speakeasy**
5. **Do not use CryptoJS**
6. **Do not use frontend frameworks**
7. **Do not use ORMs**
8. **Do not use localStorage for app data**
9. **Do not use complex encryption**
10. **Do not use unnecessary third-party libraries**

Keep the project lightweight and stable.

Only allow:

- JWT in sessionStorage
- theme preference in localStorage

Required Features

1. User Authentication

Implement simple login authentication using:

- username
- password

Store users in `users` table:

- id
- username
- password_hash
- email
- role

Use:

- bcrypt for password hashing
- JWT for authentication

Seed default users on first run:

- admin / admin123 / admin
- analyst / analyst123 / analyst
- viewer / viewer123 / viewer

Roles:

- admin = full access
- analyst = create/update
- viewer = read-only

Implement:

- POST /api/login
- GET /api/me
- POST /api/logout

Store JWT in sessionStorage on frontend.

2. Incident Management (CRUD)

Create `incidents` table:

- id
- name

- type
- severity
- status
- visibility
- date_reported
- updated_at
- notes
- created_by
- attachments (JSON string)

Implement:

- GET /api/incidents
- POST /api/incidents
- PUT /api/incidents/:id
- DELETE /api/incidents/:id (admin only)
- DELETE /api/incidents/batch (admin only)

Include filtering:

- search
- severity
- status
- visibility
- date range

Role access:

- admin: full
- analyst: create/update
- viewer: read-only

3. File Attachments

Use multer for uploads.

Requirements:

- max 5 files
- max 500KB each
- store in /uploads
- unique filenames
- save file paths in DB
- serve files statically

Frontend must support:

- attachment list
- download links
- preview if image/audio
- delete attachment (admin only)

Keep file handling simple and reliable.

4. Real-Time Alert Feed (SSE)

Use **Server-Sent Events (SSE)** only.

Implement:

- clients connect using EventSource
- backend sends alert every 15 seconds
- manual “Simulate Alert” button
- save alerts in DB

Create `alerts` table:

- id
- message
- severity
- timestamp

Frontend shows:

- live alert ticker

- recent alert panel

Keep implementation simple and stable.

5. Internal Messaging

Create `messages` table:

- id
- subject
- content
- sender_id
- recipient_id
- timestamp

Users can:

- send direct message
- send broadcast message
- view inbox

No encryption required.

Keep messaging simple and stored in database.

Frontend:

- compose form
- inbox list
- view message modal

6. Audit Logs

Create `audit_logs` table:

- id
- action
- details
- user_id
- timestamp

Log important actions:

- login
- logout
- create/update/delete incident
- batch delete
- export/import
- alert generation
- message actions

Frontend:

- show recent 5 logs
- full audit modal
- admin can clear logs

7. Export / Import

Admin only:

- export incidents as JSON
- export incidents as CSV
- import incidents from JSON (replace all)

Implement:

- GET `/api/export/json`
- GET `/api/export/csv`
- POST `/api/import/json`

Frontend:

- import/export buttons
- browser print for PDF

Keep export/import simple and reliable.

8. Theme + Responsive UI

- dark/light mode
- responsive layout
- clean dashboard
- mobile-friendly
- CSS variables

Keep UI simple, clean, and easy to use.

9. Role-Based Access Control

Implement middleware for:

- auth required
- admin only
- analyst/admin
- viewer read-only

Keep middleware simple and reusable.

10. Project Structure

Generate all files:

- server.js
- package.json
- .env.example
- database.js
- middleware/auth.js
- routes/auth.js
- routes/incidents.js
- routes/messages.js
- routes/alerts.js
- routes/audit.js
- routes/export.js
- public/index.html
- public/style.css
- public/script.js
- uploads/ (folder)
- README.md

Environment Variables

Use .env.example with:

```
JWT_SECRET=JWT_SECRET=d213f83239db76ed1172091bacc5340ebb47c348e5c69b7d78c3c89c0fc5fd9e7d94452c75d8218027a87263c0392bcb87ca78ae95499836dcad03e1949eeb0d
```

PORT=3000

BASE_URL=<http://localhost:3000>

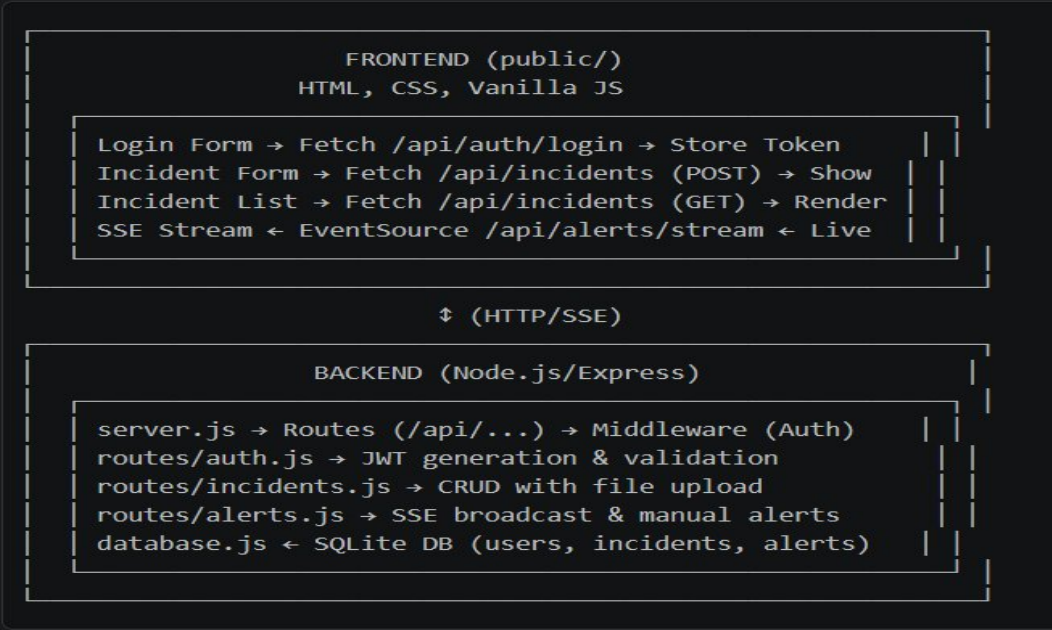
Output Rules

- Generate complete working code for all files
- Code must be fully commented
- Code must be beginner-friendly
- Keep backend simple and modular
- Avoid unnecessary abstractions
- Avoid fragile dependencies
- Must run smoothly after:
 1. npm install
 2. create .env
 3. node server.js

The final result should be a **simple, stable, effective, and easy-to-run full-stack community safety tracker** with clean backend logic and minimal setup friction

How to run : npm install
 Node server.js
 <http://localhost:3000>

6. Data Flow Diagram



How UI Connects with Backend

The frontend and backend communicate through **REST APIs** and **Server-Sent Events (SSE)** for real-time updates. Here's how:

7. Connection Summary

Type	Frontend	Backend	Example
REST GET	<code>fetch('/api/incidents', {headers})</code>	<code>router.get('/', requireAuth)</code>	Fetch incidents list
REST POST	<code>fetch('/api/incidents', {method: 'POST', body: JSON})</code>	<code>router.post('/', requireAuth, upload)</code>	Create incident
REST PUT	<code>fetch('/api/incidents/:id', {method: 'PUT'})</code>	<code>router.put('/:id', requireAuth)</code>	Update incident
REST DELETE	<code>fetch('/api/incidents/:id', {method: 'DELETE'})</code>	<code>router.delete('/:id', requireAdmin)</code>	Delete incident
File Upload	<code>FormData</code> + <code>append('file')</code>	<code>multer.array()</code>	Upload attachments
Real-Time	<code>EventSource('/api/alerts/stream')</code>	<code>res.write('data:...\\n\\n')</code>	Live alerts

Every API request includes the token in the `Authorization: Bearer <token>` header, which the backend middleware (`requireAuth`) validates before processing.

Team Member 4 — Frontend & Integration Lead
Role name: Frontend Engineer / Integration Coordinator

Main responsibilities:
Build dashboard UI and forms
Connect frontend to backend APIs
Manage role-based UI behavior
Drive integration, testing, and documentation

Technologies/tools:
HTML, CSS, Vanilla JS
Browser DevTools
GitHub workflow
Documentation tools

Estimated workload: 25%

Deliverables:
index.html
style.css
script.js
integration report
PPT slides and README updates

Assigned to: Team Member 4
Purpose: Provide the user interface, integrate modules, and document workflows.
Input: API responses, user actions
Output: visual dashboard, modals, navigation, notifications

Important functions/classes:
checkAuth()

```
// --- Authentication ---
async function checkAuth() {
  const token = sessionStorage.getItem('token');
  if (!token) {
    showLogin();
    return;
  }

  try {
    const res = await fetch('/api/auth/me', { headers: getAuthHeaders() });
    if (res.ok) {
      const data = await res.json();
      currentUser = data.user;
      showDashboard();
    } else {
      sessionStorage.removeItem('token');
      showLogin();
    }
  } catch (err) {
    showLogin();
  }
}
```

showDashboard()

```
function showDashboard() {
```

```

loginView.classList.add('hidden');
loginView.classList.remove('active-view');
dashboardView.classList.remove('hidden');
dashboardView.classList.add('active-view');

// Set user info
document.getElementById('user-avatar').textContent = currentUser.username.charAt(0).toUpperCase();
document.getElementById('user-name-display').textContent = currentUser.username;
document.getElementById('user-role-display').textContent = currentUser.role;

// Handle Role UI
document.querySelectorAll('.admin-only').forEach(el => {
    el.style.display = currentUser.role === 'admin' ? '' : 'none';
});
document.querySelectorAll('.analyst-admin-only').forEach(el => {
    el.style.display = (currentUser.role === 'admin' || currentUser.role === 'analyst') ? '' :
'none';
});

loadSection('section-incidents');
setupSSE();
}

```

fetchIncidents()

```

// --- Incidents ---
async function fetchIncidents() {
    const search = document.getElementById('search-incidents').value;
    const severity = document.getElementById('filter-severity').value;
    const status = document.getElementById('filter-status').value;

    let url =
`/api/incidents?search=${encodeURIComponent(search)}&severity=${severity}&status=${status}`;

    try {
        const res = await fetch(url, { headers: getAuthHeaders() });
        const incidents = await res.json();
        renderIncidents(incidents);
    } catch (err) {
        showToast('Failed to load incidents', 'error');
    }
}

```

handleIncidentSubmit()

```

async function handleIncidentSubmit(e) {
    e.preventDefault();
    const form = e.target;
    const id = document.getElementById('incident-id').value;
    const isUpdate = !!id;

    const formData = new FormData();
    formData.append('name', document.getElementById('inc-name').value);
    formData.append('type', document.getElementById('inc-type').value);
    formData.append('severity', document.getElementById('inc-severity').value);
    formData.append('status', document.getElementById('inc-status').value);
}

```

```

formData.append('visibility', document.getElementById('inc-visibility').value);

// Add :00 Z to make it full ISO string if needed
const d = new Date(document.getElementById('inc-date').value);
formData.append('date_reported', d.toISOString());

formData.append('notes', document.getElementById('inc-notes').value);

if (isUpdate) {
  formData.append('existing_attachments', form.dataset.existingAttachments);
}

const files = document.getElementById('inc-attachments').files;
for (let i = 0; i < files.length; i++) {
  formData.append('attachments', files[i]);
}

try {
  const url = isUpdate ? `/api/incidents/${id}` : '/api/incidents';
  const method = isUpdate ? 'PUT' : 'POST';
  const res = await fetch(url, {
    method,
    headers: getAuthHeaders(true),
    body: formData
  });

  if (res.ok) {
    showToast(isUpdate ? 'Incident updated' : 'Incident created');
    document.getElementById('incident-modal').classList.add('hidden');
    fetchIncidents();
  } else {
    const data = await res.json();
    showToast(data.error || 'Failed to save', 'error');
  }
} catch (err) {
  showToast('Connection error', 'error');
}
}

```

setupSSE()

```

// --- SSE Setup ---
function setupSSE() {
  if (sseSource) sseSource.close();
  sseSource = new EventSource('/api/alerts/stream');

  sseSource.onmessage = (event) => {
    const data = JSON.parse(event.data);
    if (data.connected) return;

    // Show Ticker
    const ticker = document.getElementById('alert-ticker');
    const tickerText = document.getElementById('latest-alert-text');
    tickerText.textContent = `${data.severity.toUpperCase(): ${data.message} (${new
Date(data.timestamp).toLocaleTimeString())}`;

```

```

    ticker.classList.remove('hidden');

    // Refresh alerts history if on that page
    if (!document.getElementById('section-alerts').classList.contains('hidden')) {
        fetchAlertsHistory();
    }
};
}

```

fetchMessages()

```

// --- Messages ---
async function fetchMessages() {
    try {
        const res = await fetch('/api/messages/inbox', { headers: getAuthHeaders() });
        const msgs = await res.json();
        const list = document.getElementById('inbox-list');
        list.innerHTML = '';

        if (msgs.length === 0) {
            list.innerHTML = '<p class="text-muted">Inbox is empty.</p>';
            return;
        }

        msgs.forEach(msg => {
            const item = document.createElement('div');
            item.className = 'list-item';
            item.innerHTML = `
                <div class="list-item-header">
                    <div class="list-item-title">${msg.subject}</div>
                    <div class="badge ${!msg.recipient_id ? 'status-open' : ''}>${!msg.recipient_id ?
'BROADCAST' : 'DIRECT'}</div>
                </div>
                <div class="list-item-meta">From: ${msg.sender_name} | ${new
Date(msg.timestamp).toLocaleString()}</div>
            `;
            item.addEventListener('click', () => {
                document.getElementById('view-msg-subject').textContent = msg.subject;
                document.getElementById('view-msg-sender').textContent = `From: ${msg.sender_name}`;
                document.getElementById('view-msg-date').textContent = new
Date(msg.timestamp).toLocaleString();
                document.getElementById('view-msg-content').textContent = msg.content;
                document.getElementById('view-message-modal').classList.remove('hidden');
            });
            list.appendChild(item);
        });
    } catch (err) {
        showToast('Failed to load messages', 'error');
    }
}

```

handleImport()

```

// --- Data Import ---
async function handleImport(e) {
    e.preventDefault();
    const file = document.getElementById('import-file').files[0];
}

```

```

if (!file) return;

const reader = new FileReader();
reader.onload = async (e) => {
  try {
    const data = JSON.parse(e.target.result);
    const res = await fetch('/api/export/json', { // It's mapped to POST /api/import/json
conceptually, let's fix the route name. Wait, the route is POST /api/import/json!
      method: 'POST',
      headers: getAuthHeaders(),
      body: JSON.stringify(data)
    });
    if (res.ok) {
      showToast('Import successful. Please wait while the page reloads.');
```

```

      setTimeout(() => location.reload(), 2000);
    } else {
      showToast('Import failed', 'error');
    }
  } catch (err) {
    showToast('Invalid JSON file', 'error');
  }
};
reader.readAsText(file);
}

```

Files/folders:

index.html
style.css
script.js

Dependencies:

All backend modules
Static asset server in server.js
Authentication token flow

Presentation/Viva Script

Introduction (Member 1) -> Akhil Nautiyal

"Hello, I'm [Name], Backend Lead. Today we present the Community Safety and Incident Tracker."
"The app supports login, incident reporting, live alerts, messaging, and audit control."
"My part covers authentication, authorization, and database design."

Problem statement (Member 1)

"Communities need a centralized system to report and manage incidents securely."
"Our solution prevents unauthorized access while preserving transparency."

Technologies used (Member 1)

"We used Node.js and Express for the backend, SQLite for storage, and JSON Web Tokens for auth."

Module explanation (Member 2) -> Anuj Dobhal

"I handled Incident Management."
"This includes the incident CRUD API and attachment workflow."
"Important code: POST /api/incidents, PUT /api/incidents/:id, and the multer upload pipeline."

Demonstration flow (Member 2)

"I'll show how an analyst creates an incident, attaches files, and updates status."

"Then I'll show how a viewer only sees public incidents."

Challenges (Member 2)

"Managing attachments safely was complex."

"We solved it by limiting upload size and saving metadata in JSON."

Module explanation (Member 3) -> Priyanshu Rawat

"I built real-time alerts, messaging, export/import, and audit logging."

"The alert stream uses SSE and admin simulation endpoints."

"Important code: server.js SSE setup, alerts.js, and export.js."

Demonstration flow (Member 3)

"I'll demonstrate live alerts appearing automatically and via manual simulate."

"Then I'll show the audit log and data export/import feature."

Challenges (Member 3)

"SSE reliability and export formatting were the hardest parts."

"We added fallback logging and carefully escaped CSV values."

Module explanation (Member 4)

"I managed the frontend and integration."

"The UI connects all back-end APIs with a single-page dashboard."

"Important code: script.js navigation, auth flow, and data binding."

Demonstration flow (Member 4)

"I'll show login, section navigation, report incident modal, inbox, and admin panels."

"Next I'll show the theme toggle and live alert ticker."

Challenges (Member 4)

"Keeping the interface responsive and role-aware required careful DOM logic."

"We used role checks in both frontend and backend for secure UX."

Conclusion (All together)

"This application provides a secure and complete incident tracking experience."

"The team delivered backend APIs, real-time alerts, messaging, and a cohesive frontend."

"We are ready for deployment and future extension."

Module 4: Frontend & Integration

Core algorithms: state management in currentUser, dynamic DOM injection for lists/tables, conditional role-based rendering.

Important functions: checkAuth(), showDashboard(), fetchIncidents(), setupSSE().

Database logic: none direct, but frontend maps API JSON to UI.

APIs used: all /api/... endpoints.

Security features: token stored in sessionStorage, Authorization header on each fetch.

Optimization techniques: reuse fetch helpers, minimal DOM updates, form reset patterns.

```

<!DOCTYPE html>
<html lang="en" data-theme="light">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Community Safety & Incident Tracker</title>
  <link rel="stylesheet" href="style.css">
  <link
href="https://fonts.googleapis.com/css2?family=Inter:wght@300;400;500;600;700&display=sw
ap" rel="stylesheet">
</head>
<body>

  <!-- Notification Toast Container -->
  <div id="toast-container"></div>

  <!-- Live Alert Ticker -->
  <div id="alert-ticker" class="hidden">
    <div class="ticker-wrap">
      <div class="ticker-heading">LIVE ALERTS:</div>
      <div class="ticker-item" id="latest-alert-text">Waiting for alerts...</div>
      <button id="close-ticker">&times;</button>
    </div>
  </div>

  <!-- Login View -->
  <div id="login-view" class="view active-view">
    <div class="login-container">
      <div class="login-card glass-panel">
        <div class="login-header">
          <h2>Community Safety</h2>
          <p>Sign in to your account</p>
        </div>
        <form id="login-form">
          <div class="form-group">
            <label for="username">Username</label>
            <input type="text" id="username" required placeholder="Enter
your username">
          </div>
          <div class="form-group">
            <label for="password">Password</label>
            <input type="password" id="password" required placeholder="Enter
your password">
          </div>
          <button type="submit" class="btn btn-primary btn-block">Log
In</button>
        </form>
        <div class="login-hint">
          <small>Default accounts: admin/admin123, analyst/analyst123,

```

```

viewer/viewer123</small>
    </div>
</div>
</div>
</div>

<!-- Main Dashboard View -->
<div id="dashboard-view" class="view hidden">
    <nav class="sidebar">
        <div class="sidebar-header">
            <h3>Safety Tracker</h3>
            <button id="theme-toggle" class="icon-btn" title="Toggle Dark/Light
Mode">☾</button>
        </div>

        <div class="user-profile">
            <div class="avatar" id="user-avatar">U</div>
            <div class="user-info">
                <div id="user-name-display" class="user-name">User Name</div>
                <div id="user-role-display" class="user-role badge">Role</div>
            </div>
        </div>

        <ul class="nav-links">
            <li><a href="#" data-target="section-incidents" class="nav-active">📄
Incidents</a></li>
            <li><a href="#" data-target="section-messages">✉ Messages</a></li>
            <li><a href="#" data-target="section-alerts">⚠ Alerts</a></li>
            <li class="admin-only"><a href="#" data-target="section-audit">🔍 Audit
Logs</a></li>
            <li class="admin-only"><a href="#" data-target="section-settings">⚙
Settings (Data)</a></li>
        </ul>

        <div class="sidebar-footer">
            <button id="logout-btn" class="btn btn-secondary btn-block">Log
Out</button>
        </div>
    </nav>

    <main class="main-content">

        <!-- Incidents Section -->
        <section id="section-incidents" class="content-section active-section">
            <div class="section-header">
                <h2>Incident Management</h2>
                <button id="btn-new-incident" class="btn btn-primary analyst-admin-
only">+ Report Incident</button>
            </div>

            <div class="filters glass-panel">
                <input type="text" id="search-incidents" placeholder="Search
incidents...">

```

```

        <select id="filter-severity">
            <option value="">All Severities</option>
            <option value="low">Low</option>
            <option value="medium">Medium</option>
            <option value="high">High</option>
            <option value="critical">Critical</option>
        </select>
        <select id="filter-status">
            <option value="">All Statuses</option>
            <option value="open">Open</option>
            <option value="in_progress">In Progress</option>
            <option value="resolved">Resolved</option>
        </select>
        <button id="btn-batch-delete" class="btn btn-danger admin-only
hidden">Batch Delete</button>
    </div>

    <div class="table-container glass-panel">
        <table id="incidents-table">
            <thead>
                <tr>
                    <th class="admin-only"><input type="checkbox"
id="select-all-incidents"></th>
                    <th>Name</th>
                    <th>Type</th>
                    <th>Severity</th>
                    <th>Status</th>
                    <th>Date Reported</th>
                    <th>Actions</th>
                </tr>
            </thead>
            <tbody id="incidents-tbody">
                <!-- Rows injected by JS -->
            </tbody>
        </table>
    </div>
</section>

<!-- Messages Section -->
<section id="section-messages" class="content-section hidden">
    <div class="section-header">
        <h2>Internal Messaging</h2>
        <button id="btn-compose-message" class="btn btn-primary">Compose
Message</button>
    </div>

    <div class="messages-container glass-panel">
        <h3>Inbox</h3>
        <div id="inbox-list" class="list-group">
            <!-- Messages injected by JS -->
        </div>
    </div>
</section>

```

```

<!-- Alerts Section -->
<section id="section-alerts" class="content-section hidden">
  <div class="section-header">
    <h2>Recent Alerts</h2>
    <button id="btn-simulate-alert" class="btn btn-warning admin-only">Simulate Alert</button>
  </div>

  <div class="alerts-container glass-panel">
    <div id="alerts-list" class="list-group">
      <!-- Alerts injected by JS -->
    </div>
  </div>
</section>

<!-- Audit Logs Section -->
<section id="section-audit" class="content-section hidden admin-only">
  <div class="section-header">
    <h2>Audit Logs</h2>
    <button id="btn-clear-logs" class="btn btn-danger">Clear
Logs</button>
  </div>

  <div class="table-container glass-panel">
    <table id="audit-table">
      <thead>
        <tr>
          <th>Time</th>
          <th>User</th>
          <th>Action</th>
          <th>Details</th>
        </tr>
      </thead>
      <tbody id="audit-tbody">
        <!-- Rows injected by JS -->
      </tbody>
    </table>
  </div>
</section>

<!-- Settings (Data) Section -->
<section id="section-settings" class="content-section hidden admin-only">
  <div class="section-header">
    <h2>Data Management</h2>
  </div>

  <div class="settings-grid">
    <div class="settings-card glass-panel">
      <h3>Export Data</h3>
      <p>Download all incidents data.</p>
      <div class="action-buttons">
        <button id="btn-export-json" class="btn btn-

```

```

secondary">Export JSON</button>
                <button id="btn-export-csv" class="btn btn-secondary">Export
CSV</button>

                </div>
            </div>

            <div class="settings-card glass-panel">
                <h3>Import Data</h3>
                <p class="text-danger">Warning: Importing JSON will replace all
existing incident records.</p>
                <form id="import-form">
                    <input type="file" id="import-file" accept=".json" required>
                    <button type="submit" class="btn btn-danger">Import &
Replace</button>

                </form>
            </div>
        </div>
    </div>
</section>

</main>
</div>

<!-- Modals -->

<!-- Incident Modal -->
<div id="incident-modal" class="modal hidden">
    <div class="modal-content glass-panel">
        <div class="modal-header">
            <h3 id="incident-modal-title">Report Incident</h3>
            <button class="close-modal">&times;</button>
        </div>
        <form id="incident-form">
            <input type="hidden" id="incident-id">
            <div class="form-row">
                <div class="form-group">
                    <label>Name</label>
                    <input type="text" id="inc-name" required>
                </div>
                <div class="form-group">
                    <label>Type</label>
                    <select id="inc-type" required>
                        <option value="theft">Theft</option>
                        <option value="vandalism">Vandalism</option>
                        <option value="accident">Accident</option>
                        <option value="suspicious">Suspicious Activity</option>
                        <option value="other">Other</option>
                    </select>
                </div>
            </div>
            <div class="form-row">
                <div class="form-group">
                    <label>Severity</label>
                    <select id="inc-severity" required>

```

```

        <option value="low">Low</option>
        <option value="medium">Medium</option>
        <option value="high">High</option>
        <option value="critical">Critical</option>
    </select>
</div>
<div class="form-group">
    <label>Status</label>
    <select id="inc-status" required>
        <option value="open">Open</option>
        <option value="in_progress">In Progress</option>
        <option value="resolved">Resolved</option>
    </select>
</div>
</div>
<div class="form-row">
    <div class="form-group">
        <label>Visibility</label>
        <select id="inc-visibility" required>
            <option value="public">Public</option>
            <option value="internal">Internal (Analyst/Admin
only)</option>
        </select>
    </div>
    <div class="form-group">
        <label>Date Reported</label>
        <input type="datetime-local" id="inc-date" required>
    </div>
</div>
<div class="form-group">
    <label>Notes</label>
    <textarea id="inc-notes" rows="4"></textarea>
</div>
<div class="form-group">
    <label>Attachments (Max 5, 500KB each)</label>
    <input type="file" id="inc-attachments" multiple
accept="image/*,audio/*,.pdf,.doc,.docx">
    <div id="existing-attachments" class="attachments-list"></div>
</div>
<div class="modal-footer">
    <button type="button" class="btn btn-secondary close-
modal">Cancel</button>
    <button type="submit" class="btn btn-primary" id="btn-save-
incident">Save Incident</button>
</div>
</form>
</div>
</div>

<!-- Compose Message Modal -->
<div id="message-modal" class="modal hidden">
    <div class="modal-content glass-panel">
        <div class="modal-header">

```

```

        <h3>Compose Message</h3>
        <button class="close-modal">&times;</button>
    </div>
    <form id="message-form">
        <div class="form-group">
            <label>To</label>
            <select id="msg-recipient" required>
                <option value="broadcast">All Users (Broadcast)</option>
                <!-- Other users injected by JS -->
            </select>
        </div>
        <div class="form-group">
            <label>Subject</label>
            <input type="text" id="msg-subject" required>
        </div>
        <div class="form-group">
            <label>Content</label>
            <textarea id="msg-content" rows="6" required></textarea>
        </div>
        <div class="modal-footer">
            <button type="button" class="btn btn-secondary close-
modal">Cancel</button>
            <button type="submit" class="btn btn-primary">Send Message</button>
        </div>
    </form>
</div>
</div>

<!-- View Message Modal -->
<div id="view-message-modal" class="modal hidden">
    <div class="modal-content glass-panel">
        <div class="modal-header">
            <h3 id="view-msg-subject">Message Subject</h3>
            <button class="close-modal">&times;</button>
        </div>
        <div class="message-meta">
            <span class="badge" id="view-msg-sender">From: User</span>
            <span id="view-msg-date" class="text-muted">Date</span>
        </div>
        <div class="message-body" id="view-msg-content">
            Message content goes here...
        </div>
        <div class="modal-footer">
            <button type="button" class="btn btn-secondary close-
modal">Close</button>
        </div>
    </div>
</div>

<!-- Simulate Alert Modal (Admin Only) -->
<div id="simulate-alert-modal" class="modal hidden">
    <div class="modal-content glass-panel">
        <div class="modal-header">

```

```

        <h3>Simulate Alert</h3>
        <button class="close-modal">&times;</button>
    </div>
    <form id="simulate-alert-form">
        <div class="form-group">
            <label>Message</label>
            <input type="text" id="sim-alert-message" required placeholder="E.g.
Road closure on Main St">
        </div>
        <div class="form-group">
            <label>Severity</label>
            <select id="sim-alert-severity" required>
                <option value="low">Low</option>
                <option value="medium">Medium</option>
                <option value="high">High</option>
                <option value="critical">Critical</option>
            </select>
        </div>
        <div class="modal-footer">
            <button type="button" class="btn btn-secondary close-
modal">Cancel</button>
            <button type="submit" class="btn btn-warning">Trigger Alert</button>
        </div>
    </form>
</div>
</div>

<!-- View Incident Attachments Modal -->
<div id="attachments-modal" class="modal hidden">
    <div class="modal-content glass-panel">
        <div class="modal-header">
            <h3>Attachments</h3>
            <button class="close-modal">&times;</button>
        </div>
        <div id="view-attachments-list" class="attachments-grid">
            <!-- Attachments injected by JS -->
        </div>
        <div class="modal-footer">
            <button type="button" class="btn btn-secondary close-
modal">Close</button>
        </div>
    </div>
</div>

<script src="script.js"></script>
</body>
</html>

```

Style.css

```

:root {
    /* Light Theme */

```

```

--bg-base: #f4f6f8;
--bg-panel: #ffffff;
--text-main: #1f2937;
--text-muted: #6b7280;
--primary: #3b82f6;
--primary-hover: #2563eb;
--secondary: #e5e7eb;
--secondary-hover: #d1d5db;
--danger: #ef4444;
--danger-hover: #dc2626;
--warning: #f59e0b;
--warning-hover: #d97706;
--border: #e5e7eb;
--glass-bg: rgba(255, 255, 255, 0.9);
--glass-border: rgba(255, 255, 255, 0.2);
--shadow: 0 4px 6px -1px rgba(0, 0, 0, 0.1), 0 2px 4px -1px rgba(0, 0, 0, 0.06);
--sidebar-width: 260px;
--radius: 12px;
}

[data-theme="dark"] {
  --bg-base: #111827;
  --bg-panel: #1f2937;
  --text-main: #f9fafb;
  --text-muted: #9ca3af;
  --primary: #3b82f6;
  --primary-hover: #60a5fa;
  --secondary: #374151;
  --secondary-hover: #4b5563;
  --border: #374151;
  --glass-bg: rgba(31, 41, 55, 0.9);
  --glass-border: rgba(255, 255, 255, 0.05);
  --shadow: 0 4px 6px -1px rgba(0, 0, 0, 0.5);
}

* {
  box-sizing: border-box;
  margin: 0;
  padding: 0;
}

body {
  font-family: 'Inter', sans-serif;
  background-color: var(--bg-base);
  color: var(--text-main);
  transition: background-color 0.3s, color 0.3s;
  overflow-x: hidden;
}

/* Typography & Utilities */
h1, h2, h3 { font-weight: 600; }
a { text-decoration: none; color: var(--primary); }
.hidden { display: none !important; }

```

```

.text-muted { color: var(--text-muted); }
.text-danger { color: var(--danger); }

/* Buttons */
.btn {
  display: inline-flex;
  align-items: center;
  justify-content: center;
  padding: 0.5rem 1rem;
  font-weight: 500;
  border-radius: 6px;
  border: none;
  cursor: pointer;
  transition: all 0.2s;
  font-size: 0.9rem;
}
.btn-primary { background-color: var(--primary); color: white; }
.btn-primary:hover { background-color: var(--primary-hover); }
.btn-secondary { background-color: var(--secondary); color: var(--text-main); }
.btn-secondary:hover { background-color: var(--secondary-hover); }
.btn-danger { background-color: var(--danger); color: white; }
.btn-danger:hover { background-color: var(--danger-hover); }
.btn-warning { background-color: var(--warning); color: white; }
.btn-warning:hover { background-color: var(--warning-hover); }
.btn-block { width: 100%; }
.icon-btn { background: none; border: none; cursor: pointer; font-size: 1.2rem; color: var(--text-main); }

/* Badges */
.badge {
  display: inline-block;
  padding: 0.25rem 0.5rem;
  border-radius: 9999px;
  font-size: 0.75rem;
  font-weight: 600;
  background: var(--secondary);
  color: var(--text-main);
  text-transform: uppercase;
  letter-spacing: 0.05em;
}
.badge.severity-low { background: #10b981; color: white; }
.badge.severity-medium { background: var(--warning); color: white; }
.badge.severity-high, .badge.severity-critical { background: var(--danger); color: white; }
.badge.status-open { background: var(--warning); color: white; }
.badge.status-in_progress { background: var(--primary); color: white; }
.badge.status-resolved { background: #10b981; color: white; }

/* Forms */
.form-group { margin-bottom: 1rem; }
.form-row { display: flex; gap: 1rem; }
.form-row .form-group { flex: 1; }
label { display: block; margin-bottom: 0.5rem; font-weight: 500; font-size: 0.9rem; }

```

```

input[type="text"], input[type="password"], input[type="datetime-local"], select,
textarea {
  width: 100%;
  padding: 0.75rem;
  border: 1px solid var(--border);
  border-radius: 6px;
  background-color: var(--bg-base);
  color: var(--text-main);
  font-family: inherit;
  transition: border-color 0.2s;
}
input:focus, select:focus, textarea:focus {
  outline: none;
  border-color: var(--primary);
  box-shadow: 0 0 0 3px rgba(59, 130, 246, 0.1);
}

/* Glass Panel */
.glass-panel {
  background: var(--glass-bg);
  backdrop-filter: blur(10px);
  -webkit-backdrop-filter: blur(10px);
  border: 1px solid var(--glass-border);
  border-radius: var(--radius);
  box-shadow: var(--shadow);
}

/* Views */
.view {
  min-height: 100vh;
  width: 100%;
}
#login-view {
  display: flex;
  align-items: center;
  justify-content: center;
  background: linear-gradient(135deg, var(--bg-base) 0%, var(--border) 100%);
}
.login-card {
  width: 100%;
  max-width: 400px;
  padding: 2.5rem;
}
.login-header { text-align: center; margin-bottom: 2rem; }
.login-hint { text-align: center; margin-top: 1.5rem; color: var(--text-muted); }

/* Dashboard Layout */
#dashboard-view {
  display: flex;
}
.sidebar {
  width: var(--sidebar-width);
  height: 100vh;

```

```
position: fixed;
top: 0;
left: 0;
background: var(--bg-panel);
border-right: 1px solid var(--border);
display: flex;
flex-direction: column;
z-index: 100;
}
.sidebar-header {
padding: 1.5rem;
display: flex;
justify-content: space-between;
align-items: center;
border-bottom: 1px solid var(--border);
}
.user-profile {
padding: 1.5rem;
display: flex;
align-items: center;
gap: 1rem;
border-bottom: 1px solid var(--border);
}
.avatar {
width: 40px;
height: 40px;
border-radius: 50%;
background: var(--primary);
color: white;
display: flex;
align-items: center;
justify-content: center;
font-weight: bold;
font-size: 1.2rem;
}
.nav-links {
list-style: none;
padding: 1rem 0;
flex: 1;
}
.nav-links li a {
display: block;
padding: 0.75rem 1.5rem;
color: var(--text-muted);
font-weight: 500;
transition: all 0.2s;
}
.nav-links li a:hover, .nav-links li a.nav-active {
background: var(--secondary);
color: var(--primary);
border-right: 4px solid var(--primary);
}
.sidebar-footer { padding: 1.5rem; border-top: 1px solid var(--border); }
```

```

.main-content {
  flex: 1;
  margin-left: var(--sidebar-width);
  padding: 2rem;
  min-height: 100vh;
}

.section-header {
  display: flex;
  justify-content: space-between;
  align-items: center;
  margin-bottom: 2rem;
}

/* Tables & Lists */
.filters {
  padding: 1rem;
  display: flex;
  gap: 1rem;
  margin-bottom: 1rem;
  flex-wrap: wrap;
  align-items: center;
}

.filters input { flex: 1; min-width: 200px; }
.table-container {
  overflow-x: auto;
}

table {
  width: 100%;
  border-collapse: collapse;
}

th, td {
  padding: 1rem;
  text-align: left;
  border-bottom: 1px solid var(--border);
}

th { font-weight: 600; color: var(--text-muted); background: var(--bg-panel); }
tr:hover { background-color: var(--bg-base); }
.actions-cell { display: flex; gap: 0.5rem; }

/* Lists (Messages/Alerts) */
.list-group { display: flex; flex-direction: column; gap: 0.5rem; padding: 1rem; }
.list-item {
  padding: 1rem;
  border: 1px solid var(--border);
  border-radius: 8px;
  background: var(--bg-panel);
  transition: transform 0.2s, box-shadow 0.2s;
  cursor: pointer;
}

.list-item:hover {
  transform: translateY(-2px);
  box-shadow: var(--shadow);
}

```

```

    border-color: var(--primary);
}
.list-item-header { display: flex; justify-content: space-between; margin-bottom:
0.5rem; }
.list-item-title { font-weight: 600; }
.list-item-meta { font-size: 0.8rem; color: var(--text-muted); }

/* Modals */
.modal {
    position: fixed;
    top: 0; left: 0; right: 0; bottom: 0;
    background: rgba(0,0,0,0.5);
    display: flex;
    align-items: center;
    justify-content: center;
    z-index: 1000;
    padding: 1rem;
}
.modal-content {
    width: 100%;
    max-width: 600px;
    max-height: 90vh;
    overflow-y: auto;
    padding: 2rem;
}
.modal-header { display: flex; justify-content: space-between; align-items: center;
margin-bottom: 1.5rem; }
.close-modal { background: none; border: none; font-size: 1.5rem; cursor: pointer;
color: var(--text-muted); }
.modal-footer { margin-top: 2rem; display: flex; justify-content: flex-end; gap: 1rem; }

/* Toast */
#toast-container {
    position: fixed; bottom: 20px; right: 20px; z-index: 9999;
    display: flex; flex-direction: column; gap: 10px;
}
.toast {
    padding: 1rem 1.5rem;
    background: var(--bg-panel);
    color: var(--text-main);
    border-radius: 8px;
    box-shadow: var(--shadow);
    border-left: 4px solid var(--primary);
    animation: slideIn 0.3s ease-out forwards;
}
.toast.success { border-color: #10b981; }
.toast.error { border-color: var(--danger); }
@keyframes slideIn {
    from { transform: translateX(100%); opacity: 0; }
    to { transform: translateX(0); opacity: 1; }
}

/* Alert Ticker */

```

```

#alert-ticker {
  position: fixed; top: 0; left: 0; width: 100%;
  background: var(--danger); color: white;
  z-index: 2000; padding: 0.5rem 1rem;
}
.ticker-wrap { display: flex; align-items: center; gap: 1rem; }
.ticker-heading { font-weight: bold; white-space: nowrap; }
.ticker-item { flex: 1; overflow: hidden; white-space: nowrap; text-overflow:
ellipsis; }
#close-ticker { background: none; border: none; color: white; font-size: 1.2rem; cursor:
pointer; }

/* Settings Grid */
.settings-grid { display: grid; grid-template-columns: repeat(auto-fit, minmax(300px,
1fr)); gap: 1.5rem; }
.settings-card { padding: 1.5rem; }
.settings-card h3 { margin-bottom: 0.5rem; }
.settings-card p { margin-bottom: 1.5rem; color: var(--text-muted); }
.action-buttons { display: flex; gap: 1rem; }

/* Attachments */
.attachments-grid { display: grid; grid-template-columns: repeat(auto-fill,
minmax(120px, 1fr)); gap: 1rem; }
.attachment-card { text-align: center; padding: 0.5rem; border: 1px solid var(--border);
border-radius: 8px; }
.attachment-card img { max-width: 100%; border-radius: 4px; margin-bottom: 0.5rem; }
.attachment-card a { font-size: 0.8rem; word-break: break-all; }

/* Responsive */
@media (max-width: 768px) {
  .sidebar { transform: translateX(-100%); transition: transform 0.3s; }
  .sidebar.open { transform: translateX(0); }
  .main-content { margin-left: 0; }
  .form-row { flex-direction: column; gap: 0; }
  /* We'd need a hamburger menu button, handled in JS if needed, but keeping it simple
for now */
}

```

Script.js

```

// --- Global State ---
let currentUser = null;
let sseSource = null;

// --- DOM Elements ---
const loginView = document.getElementById('login-view');
const dashboardView = document.getElementById('dashboard-view');
const themeToggle = document.getElementById('theme-toggle');

// --- Initialization ---
document.addEventListener('DOMContentLoaded', () => {
  initTheme();

```

```

    checkAuth();
    setupEventListeners();
  });

// --- Theme ---
function initTheme() {
  const savedTheme = localStorage.getItem('theme') || 'light';
  document.documentElement.setAttribute('data-theme', savedTheme);
  themeToggle.textContent = savedTheme === 'dark' ? '☀' : '🌙';
}

themeToggle.addEventListener('click', () => {
  const currentTheme = document.documentElement.getAttribute('data-theme');
  const newTheme = currentTheme === 'light' ? 'dark' : 'light';
  document.documentElement.setAttribute('data-theme', newTheme);
  localStorage.setItem('theme', newTheme);
  themeToggle.textContent = newTheme === 'dark' ? '☀' : '🌙';
});

// --- Utilities ---
function showToast(message, type = 'success') {
  const container = document.getElementById('toast-container');
  const toast = document.createElement('div');
  toast.className = `toast ${type}`;
  toast.textContent = message;
  container.appendChild(toast);
  setTimeout(() => {
    toast.remove();
  }, 3000);
}

function getAuthHeaders(isFileUpload = false) {
  const token = sessionStorage.getItem('token');
  const headers = { 'Authorization': `Bearer ${token}` };
  if (!isFileUpload) {
    headers['Content-Type'] = 'application/json';
  }
  return headers;
}

// --- Authentication ---
async function checkAuth() {
  const token = sessionStorage.getItem('token');
  if (!token) {
    showLogin();
    return;
  }

  try {
    const res = await fetch('/api/auth/me', { headers: getAuthHeaders() });
    if (res.ok) {
      const data = await res.json();
      currentUser = data.user;
    }
  }
}

```

```

        showDashboard();
    } else {
        sessionStorage.removeItem('token');
        showLogin();
    }
} catch (err) {
    showLogin();
}
}

function showLogin() {
    loginView.classList.add('active-view');
    loginView.classList.remove('hidden');
    dashboardView.classList.add('hidden');
    dashboardView.classList.remove('active-view');
    if (sseSource) sseSource.close();
}

function showDashboard() {
    loginView.classList.add('hidden');
    loginView.classList.remove('active-view');
    dashboardView.classList.remove('hidden');
    dashboardView.classList.add('active-view');

    // Set user info
    document.getElementById('user-avatar').textContent =
currentUser.username.charAt(0).toUpperCase();
    document.getElementById('user-name-display').textContent = currentUser.username;
    document.getElementById('user-role-display').textContent = currentUser.role;

    // Handle Role UI
    document.querySelectorAll('.admin-only').forEach(el => {
        el.style.display = currentUser.role === 'admin' ? '' : 'none';
    });
    document.querySelectorAll('.analyst-admin-only').forEach(el => {
        el.style.display = (currentUser.role === 'admin' || currentUser.role ===
'analyst') ? '' : 'none';
    });

    loadSection('section-incidents');
    setupSSE();
}

document.getElementById('login-form').addEventListener('submit', async (e) => {
    e.preventDefault();
    const username = document.getElementById('username').value;
    const password = document.getElementById('password').value;

    try {
        const res = await fetch('/api/auth/login', {
            method: 'POST',
            headers: { 'Content-Type': 'application/json' },
            body: JSON.stringify({ username, password })
        });
    }

```

```

    });
    const data = await res.json();

    if (res.ok) {
        sessionStorage.setItem('token', data.token);
        currentUser = data.user;
        showDashboard();
        showToast('Logged in successfully');
    } else {
        showToast(data.error || 'Login failed', 'error');
    }
} catch (err) {
    showToast('Connection error', 'error');
}
});

document.getElementById('logout-btn').addEventListener('click', async () => {
    try {
        await fetch('/api/auth/logout', { method: 'POST', headers: getAuthHeaders() });
    } catch (e) {} // ignore errors on logout
    sessionStorage.removeItem('token');
    currentUser = null;
    showLogin();
});

// --- Navigation ---
function setupEventListeners() {
    document.querySelectorAll('.nav-links a').forEach(link => {
        link.addEventListener('click', (e) => {
            e.preventDefault();
            document.querySelectorAll('.nav-links a').forEach(l =>
l.classList.remove('nav-active'));
            link.classList.add('nav-active');
            const target = link.getAttribute('data-target');
            loadSection(target);
        });
    });
});

// Modals Close
document.querySelectorAll('.close-modal').forEach(btn => {
    btn.addEventListener('click', () => {
        btn.closest('.modal').classList.add('hidden');
    });
});

// Alert Ticker
document.getElementById('close-ticker').addEventListener('click', () => {
    document.getElementById('alert-ticker').classList.add('hidden');
});

// Incidents Setup
document.getElementById('btn-new-incident').addEventListener('click',
openIncidentModal);

```

```

    document.getElementById('incident-form').addEventListener('submit',
handleIncidentSubmit);

    // Filters
    document.getElementById('search-incidents').addEventListener('input',
fetchIncidents);
    document.getElementById('filter-severity').addEventListener('change',
fetchIncidents);
    document.getElementById('filter-status').addEventListener('change', fetchIncidents);

    // Messages Setup
    document.getElementById('btn-compose-message').addEventListener('click',
openComposeModal);
    document.getElementById('message-form').addEventListener('submit',
handleMessageSubmit);

    // Alerts Setup
    document.getElementById('btn-simulate-alert').addEventListener('click', () => {
        document.getElementById('simulate-alert-modal').classList.remove('hidden');
    });
    document.getElementById('simulate-alert-form').addEventListener('submit',
handleSimulateAlert);

    // Audit Setup
    document.getElementById('btn-clear-logs').addEventListener('click', clearAuditLogs);

    // Export/Import Setup
    document.getElementById('btn-export-json').addEventListener('click', () =>
window.open('/api/export/json?token=' + sessionStorage.getItem('token'), '_blank'));
    document.getElementById('btn-export-csv').addEventListener('click', () =>
window.open('/api/export/csv?token=' + sessionStorage.getItem('token'), '_blank'));
    document.getElementById('import-form').addEventListener('submit', handleImport);
}

function loadSection(sectionId) {
    document.querySelectorAll('.content-section').forEach(sec =>
sec.classList.add('hidden'));
    document.getElementById(sectionId).classList.remove('hidden');

    if (sectionId === 'section-incidents') fetchIncidents();
    if (sectionId === 'section-messages') fetchMessages();
    if (sectionId === 'section-alerts') fetchAlertsHistory();
    if (sectionId === 'section-audit') fetchAuditLogs();
}

// --- SSE Setup ---
function setupSSE() {
    if (sseSource) sseSource.close();
    sseSource = new EventSource('/api/alerts/stream');

    sseSource.onmessage = (event) => {
        const data = JSON.parse(event.data);
        if (data.connected) return;

```

```

    // Show Ticker
    const ticker = document.getElementById('alert-ticker');
    const tickerText = document.getElementById('latest-alert-text');
    tickerText.textContent = `${data.severity.toUpperCase():} ${data.message} (${new
Date(data.timestamp).toLocaleTimeString())}`;
    ticker.classList.remove('hidden');

    // Refresh alerts history if on that page
    if (!document.getElementById('section-alerts').classList.contains('hidden')) {
        fetchAlertsHistory();
    }
};
}

// --- Incidents ---
async function fetchIncidents() {
    const search = document.getElementById('search-incidents').value;
    const severity = document.getElementById('filter-severity').value;
    const status = document.getElementById('filter-status').value;

    let url =
`/api/incidents?search=${encodeURIComponent(search)}&severity=${severity}&status=${statu
s}`;

    try {
        const res = await fetch(url, { headers: getAuthHeaders() });
        const incidents = await res.json();
        renderIncidents(incidents);
    } catch (err) {
        showToast('Failed to load incidents', 'error');
    }
}

function renderIncidents(incidents) {
    const tbody = document.getElementById('incidents-tbody');
    tbody.innerHTML = '';

    incidents.forEach(inc => {
        const tr = document.createElement('tr');

        const attachCount = inc.attachments ? inc.attachments.length : 0;

        tr.innerHTML = `
            <td class="admin-only" style="display: ${currentUser.role === 'admin' ? '' :
'none'}"><input type="checkbox" class="incident-checkbox" value="${inc.id}"></td>
            <td><strong>${inc.name}</strong><br><small class="text-
muted">${inc.creator_name || 'Unknown'}</small></td>
            <td>${inc.type}</td>
            <td><span class="badge severity-${inc.severity}">${inc.severity}</span></td>
            <td><span class="badge status-${inc.status}">${inc.status.replace('_', '
')}}</span></td>
            <td>${new Date(inc.date_reported).toLocaleDateString()}</td>

```

```

        <td class="actions-cell">
            ${attachCount > 0 ? `<button class="btn btn-secondary btn-view-attach"
data-id="${inc.id}">0 (${attachCount})</button>` : ''}
            ${((currentUser.role === 'admin' || currentUser.role === 'analyst')) ?
`<button class="btn btn-primary btn-edit" data-id="${inc.id}">Edit</button>` : ''}
            ${currentUser.role === 'admin' ? `<button class="btn btn-danger btn-
delete" data-id="${inc.id}">Delete</button>` : ''}
        </td>
    `;
    tbody.appendChild(tr);

    // Attach data for modals
    tr.dataset.incident = JSON.stringify(inc);
});

// Edit Events
document.querySelectorAll('.btn-edit').forEach(btn => {
    btn.addEventListener('click', (e) => {
        const row = e.target.closest('tr');
        const inc = JSON.parse(row.dataset.incident);
        openIncidentModal(inc);
    });
});

// Delete Events
document.querySelectorAll('.btn-delete').forEach(btn => {
    btn.addEventListener('click', async (e) => {
        if (!confirm('Are you sure you want to delete this incident?')) return;
        const id = e.target.dataset.id;
        try {
            await fetch(`/api/incidents/${id}`, { method: 'DELETE', headers:
getAuthHeaders() });
            showToast('Incident deleted');
            fetchIncidents();
        } catch (err) { showToast('Error deleting', 'error'); }
    });
});

// View Attachments Events
document.querySelectorAll('.btn-view-attach').forEach(btn => {
    btn.addEventListener('click', (e) => {
        const row = e.target.closest('tr');
        const inc = JSON.parse(row.dataset.incident);
        openAttachmentsModal(inc.attachments);
    });
});
}

function openIncidentModal(incident = null) {
    const modal = document.getElementById('incident-modal');
    const form = document.getElementById('incident-form');
    form.reset();
    document.getElementById('existing-attachments').innerHTML = '';

```

```

if (incident && incident.id) {
  document.getElementById('incident-modal-title').textContent = 'Edit Incident';
  document.getElementById('incident-id').value = incident.id;
  document.getElementById('inc-name').value = incident.name;
  document.getElementById('inc-type').value = incident.type;
  document.getElementById('inc-severity').value = incident.severity;
  document.getElementById('inc-status').value = incident.status;
  document.getElementById('inc-visibility').value = incident.visibility;
  document.getElementById('inc-date').value = incident.date_reported.slice(0, 16);
  document.getElementById('inc-notes').value = incident.notes || '';

  // Store existing attachments to send back on update
  if (incident.attachments && incident.attachments.length > 0) {
    form.dataset.existingAttachments = JSON.stringify(incident.attachments);
    incident.attachments.forEach((att, idx) => {
      const p = document.createElement('div');
      p.textContent = `📎 ${att.originalname}`;
      document.getElementById('existing-attachments').appendChild(p);
    });
  } else {
    form.dataset.existingAttachments = '[]';
  }
} else {
  document.getElementById('incident-modal-title').textContent = 'Report Incident';
  document.getElementById('incident-id').value = '';
  form.dataset.existingAttachments = '[]';
  const now = new Date();
  now.setMinutes(now.getMinutes() - now.getTimezoneOffset());
  document.getElementById('inc-date').value = now.toISOString().slice(0, 16);
}

modal.classList.remove('hidden');
}

async function handleIncidentSubmit(e) {
  e.preventDefault();
  const form = e.target;
  const id = document.getElementById('incident-id').value;
  const isUpdate = !!id;

  const formData = new FormData();
  formData.append('name', document.getElementById('inc-name').value);
  formData.append('type', document.getElementById('inc-type').value);
  formData.append('severity', document.getElementById('inc-severity').value);
  formData.append('status', document.getElementById('inc-status').value);
  formData.append('visibility', document.getElementById('inc-visibility').value);

  // Add :00 Z to make it full ISO string if needed
  const d = new Date(document.getElementById('inc-date').value);
  formData.append('date_reported', d.toISOString());

  formData.append('notes', document.getElementById('inc-notes').value);

```

```

if (isUpdate) {
  formData.append('existing_attachments', form.dataset.existingAttachments);
}

const files = document.getElementById('inc-attachments').files;
for (let i = 0; i < files.length; i++) {
  formData.append('attachments', files[i]);
}

try {
  const url = isUpdate ? `/api/incidents/${id}` : '/api/incidents';
  const method = isUpdate ? 'PUT' : 'POST';
  const res = await fetch(url, {
    method,
    headers: getAuthHeaders(true),
    body: formData
  });

  if (res.ok) {
    showToast(isUpdate ? 'Incident updated' : 'Incident created');
    document.getElementById('incident-modal').classList.add('hidden');
    fetchIncidents();
  } else {
    const data = await res.json();
    showToast(data.error || 'Failed to save', 'error');
  }
} catch (err) {
  showToast('Connection error', 'error');
}
}

function openAttachmentsModal(attachments) {
  const list = document.getElementById('view-attachments-list');
  list.innerHTML = '';

  attachments.forEach(attach => {
    const card = document.createElement('div');
    card.className = 'attachment-card';

    let preview = '';
    if (attach.mimetype.startsWith('image/')) {
      preview = ``;
    } else {
      preview = `<div style="font-size: 2rem; margin-bottom: 0.5rem">📎</div>`;
    }

    card.innerHTML = `
      ${preview}
      <a href="${attach.path}" target="_blank"
download="${attach.originalname}">${attach.originalname}</a>
    `;
    list.appendChild(card);
  });
}

```

```

    });

    document.getElementById('attachments-modal').classList.remove('hidden');
}

// --- Messages ---
async function fetchMessages() {
    try {
        const res = await fetch('/api/messages/inbox', { headers: getAuthHeaders() });
        const msgs = await res.json();
        const list = document.getElementById('inbox-list');
        list.innerHTML = '';

        if (msgs.length === 0) {
            list.innerHTML = '<p class="text-muted">Inbox is empty.</p>';
            return;
        }

        msgs.forEach(msg => {
            const item = document.createElement('div');
            item.className = 'list-item';
            item.innerHTML = `
                <div class="list-item-header">
                    <div class="list-item-title">${msg.subject}</div>
                    <div class="badge ${!msg.recipient_id ? 'status-open' : ''}>${!msg.recipient_id ? 'BROADCAST' : 'DIRECT'}</div>
                </div>
                <div class="list-item-meta">From: ${msg.sender_name} | ${new
Date(msg.timestamp).toLocaleString()}</div>
            `;
            item.addEventListener('click', () => {
                document.getElementById('view-msg-subject').textContent = msg.subject;
                document.getElementById('view-msg-sender').textContent = `From:
${msg.sender_name}`;
                document.getElementById('view-msg-date').textContent = new
Date(msg.timestamp).toLocaleString();
                document.getElementById('view-msg-content').textContent = msg.content;
                document.getElementById('view-message-
modal').classList.remove('hidden');
            });
            list.appendChild(item);
        });
    } catch (err) {
        showToast('Failed to load messages', 'error');
    }
}

async function openComposeModal() {
    const select = document.getElementById('msg-recipient');
    select.innerHTML = '<option value="broadcast">All Users (Broadcast)</option>';

    try {
        const res = await fetch('/api/messages/users', { headers: getAuthHeaders() });

```

```

        const users = await res.json();
        users.forEach(u => {
            const opt = document.createElement('option');
            opt.value = u.id;
            opt.textContent = u.username;
            select.appendChild(opt);
        });
    } catch (err) {}

    document.getElementById('message-form').reset();
    document.getElementById('message-modal').classList.remove('hidden');
}

async function handleMessageSubmit(e) {
    e.preventDefault();
    const recipient_id = document.getElementById('msg-recipient').value;
    const subject = document.getElementById('msg-subject').value;
    const content = document.getElementById('msg-content').value;

    try {
        const res = await fetch('/api/messages', {
            method: 'POST',
            headers: getAuthHeaders(),
            body: JSON.stringify({ recipient_id, subject, content })
        });
        if (res.ok) {
            showToast('Message sent');
            document.getElementById('message-modal').classList.add('hidden');
            fetchMessages();
        } else {
            showToast('Failed to send message', 'error');
        }
    } catch (err) {
        showToast('Error sending message', 'error');
    }
}

// --- Alerts ---
async function fetchAlertsHistory() {
    try {
        const res = await fetch('/api/alerts', { headers: getAuthHeaders() });
        const alerts = await res.json();
        const list = document.getElementById('alerts-list');
        list.innerHTML = '';

        alerts.forEach(al => {
            const item = document.createElement('div');
            item.className = 'list-item';
            item.innerHTML = `
                <div class="list-item-header">
                    <div class="list-item-title">${al.message}</div>
                    <span class="badge severity-${al.severity}">${al.severity}</span>
                </div>
            `;
        });
    }
}

```

```

        <div class="list-item-meta">${new
Date(al.timestamp).toLocaleString()}</div>
        `;
        list.appendChild(item);
    });
} catch (err) {
    showToast('Failed to load alerts', 'error');
}
}

async function handleSimulateAlert(e) {
    e.preventDefault();
    const message = document.getElementById('sim-alert-message').value;
    const severity = document.getElementById('sim-alert-severity').value;

    try {
        const res = await fetch('/api/alerts/simulate', {
            method: 'POST',
            headers: getAuthHeaders(),
            body: JSON.stringify({ message, severity })
        });
        if (res.ok) {
            showToast('Alert simulated successfully');
            document.getElementById('simulate-alert-modal').classList.add('hidden');
            // SSE will trigger UI update automatically
        } else {
            showToast('Failed to simulate', 'error');
        }
    } catch (err) {
        showToast('Connection error', 'error');
    }
}

// --- Audit Logs ---
async function fetchAuditLogs() {
    try {
        const res = await fetch('/api/audit', { headers: getAuthHeaders() });
        const logs = await res.json();
        const tbody = document.getElementById('audit-tbody');
        tbody.innerHTML = '';

        logs.forEach(log => {
            const tr = document.createElement('tr');
            tr.innerHTML = `
                <td>${new Date(log.timestamp).toLocaleString()}</td>
                <td>${log.username || 'System'}</td>
                <td><span class="badge">${log.action.replace(/_/g, ' ')}</span></td>
                <td><small>${log.details}</small></td>
            `;
            tbody.appendChild(tr);
        });
    } catch (err) {
        showToast('Failed to load logs', 'error');
    }
}

```

```

    }
}

async function clearAuditLogs() {
  if (!confirm('Are you sure you want to clear all audit logs?')) return;
  try {
    const res = await fetch('/api/audit', { method: 'DELETE', headers:
getAuthHeaders() });
    if (res.ok) {
      showToast('Logs cleared');
      fetchAuditLogs();
    } else {
      showToast('Failed to clear logs', 'error');
    }
  } catch (err) {
    showToast('Connection error', 'error');
  }
}

// --- Data Import ---
async function handleImport(e) {
  e.preventDefault();
  const file = document.getElementById('import-file').files[0];
  if (!file) return;

  const reader = new FileReader();
  reader.onload = async (e) => {
    try {
      const data = JSON.parse(e.target.result);
      const res = await fetch('/api/export/json', { // It's mapped to POST
/api/import/json conceptually, let's fix the route name. Wait, the route is POST
/api/import/json!
        method: 'POST',
        headers: getAuthHeaders(),
        body: JSON.stringify(data)
      });
      if (res.ok) {
        showToast('Import successful. Please wait while the page reloads.');
```

setTimeout(() => location.reload(), 2000);

```

      } else {
        showToast('Import failed', 'error');
      }
    } catch (err) {
      showToast('Invalid JSON file', 'error');
    }
  };
  reader.readAsText(file);
}

// Fix the import fetch call above to use correct URL
document.getElementById('import-form').addEventListener('submit', async (e) => {
  e.preventDefault();
  const file = document.getElementById('import-file').files[0];

```

```
if (!file) return;

const reader = new FileReader();
reader.onload = async (evt) => {
  try {
    const data = JSON.parse(evt.target.result);
    const res = await fetch('/api/import/json', {
      method: 'POST',
      headers: getAuthHeaders(),
      body: JSON.stringify(data)
    });
    if (res.ok) {
      showToast('Import successful!');
      document.getElementById('import-form').reset();
    } else {
      showToast('Import failed', 'error');
    }
  } catch (err) {
    showToast('Invalid JSON file', 'error');
  }
};
reader.readAsText(file);
});
```